Technical Summary: AssistAnt Orchestrator Routing

Angeles University Foundation · PTF60 Project · BS Computer Science 4

Request	Write a technical summary report on how AssistAnt's Orchestrator routes work to its agents and models, including the fallback chain.
Skill	Technical report
Routing	hard task, best tier
Models	Kilo (keyless), plain rules
Review	2 round(s), approved, 19/19 claims grounded
Generated	2026-11-05 23:31

Summary

Every request goes to the Orchestrator. [S1] The Orchestrator hands work to eleven subagents. [S1] When no routine matches, the Orchestrator answers the request and drafts a reusable routine. [S2] All agents read and change data through one Workspace interface, so flows never call Notion directly. [S3]

Key findings

- The Orchestrator tests every routine's triggers against the request and runs the first routine that matches. [S5]
- Each agent has a tier that decides which line of models it asks for. [S1]
- If none matches, the Orchestrator plans the request itself with the unplanned flow and offers to save a new routine. [S1]
- A run emits events such as the plan, messages between agents, work done and the result. [S9]
- In public demo mode, the workspace is always the fake sample workspace, whatever the environment holds. [S7]
- Each browser gets a random visitor ID in a cookie, set by middleware before the first API call. [S7]

How it works

- The Notion workspace needs NOTION_TOKEN plus the Tasks page, Reminders database and Notes page IDs. [S3]
- The Stocks sheet is reached with a Google service account. [S3]
- If every provider in the chain fails or is unavailable, the router returns nothing and the agent falls back to plain rules. [S10]
- Every flow is written so it can finish without a model, which keeps AssistAnt working offline and for free. [S10]
- AssistAnt is a personal agent that wraps up the day in Notion and plans tomorrow. [S1]
- The router walks that tier's chain of providers and uses the first one that is available. [S8]

Risks and limitations

- They are off by default because the list says they may log prompts. [S10]
- It asks for approval before saving it, so a new routine never appears without the user's OK. [S2]

- Risky changes wait for the user. [S9]
- Usage is counted per provider per day and resets at midnight in the configured time zone. [S10]
- The public demo switches keyless gateways on because it only handles fake data. [S7]
- The demo workspace is a fictional user's Tasks page with sample tasks, reminders and notes. [S3]

Sources

- [S1] architecture.md, part 1
- [S2] routines.md, part 3
- [S3] workspace.md, part 1
- [S5] routines.md, part 1
- [S7] public-demo.md, part 1
- [S8] model-router.md, part 1
- [S9] architecture.md, part 3
- [S10] model-router.md, part 2

How the team worked

You → **Orchestrator** · New request
From now on, always end reports with a 'Risks and limitations' section. Write a technical summary report on how AssistAnt's Orchestrator routes work to its agents and models, including the fallback chain.

Orchestrator → **Memory** · Save standing rule
From now on, always end reports with a 'Risks and limitations' section.

Orchestrator → **Reader** · Assign the crew
Reader Notes Scout Writer Checker Publisher

Reader → **Notes Scout** · Notes to verify
20 notes from 5 files

Notes Scout → **Writer** · Evidence pack
10 passages, S1–S10

Writer → **Checker** · Draft for review
round 1

Checker → **Writer** · Send back for revision
Reader: public-demo.md was read and has evidence but is never cited. | Reader: workspace.md was read and has evidence but is never cited.

Writer → **Checker** · Draft for review
round 2

Checker → **Publisher** · Approved
approved

Publisher → **You** · Report ready
421 words, 8 sources